Lecture 04: Python Basics (Chapter 2 & 3)

교재 Ch. 2.2, 2.5, 2.6, 3.1

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 10일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

2.2 Type Conversion (형변환)

2.2 Type Conversion: 암시적 형변환 (Implicit)

- **개념:** 파이썬이 필요에 따라 자동으로 데이터 타입을 변환

자동 변환 예시: GPS 거리 계산

```
distance = 250    # 초기값: int
distance = 252.5    # 새로운 값 할당 시 자동으로 float으로 변환됨
```

파이썬은 융통성이 있어서, 정수(int) 자리에 실수(float)가 들어오면 (군말 없이) 방의 크기를 늘려줌

명시적 형변환 (Explicit Type Conversion)

- **개념:** 프로그래머가 `int()`, `float()`, `str()` 함수를 사용하여 타입을 강제로 바꾸는 것입니다.

함수	역할	예시
`int()`	정수로 변환 (소수점 아래 버림)	`int(5.9)` → 5
`float()`	실수로 변환	`float(2)` → 2.0
`str()`	문자열로 변환	`str(3.14)` → "3.14"

주의: int()는 반올림이 아닙니다!

`int(5.9)`는 6이 아니라 5가 됩니다.

실전 활용: input() 데이터 처리

input() 함수로 받은 값은 무조건 **문자열(String)**입니다. 따라서 숫자로 계산하려면 반드시 형변환이 필요합니다.

Pizza Ordering System

```
slices_str = input("How many slices?")
slices = int(slices_str)   # 명시적 형변환!
total = slices * 3000
print("Total: " + str(total) + " won")
```

2.5 Dividing Integers (나눗셈의 종류)

2.5 나눗셈 연산자 3종 세트

파이썬은 나눗셈을 목적에 따라 세 가지 방식으로 처리합니다.

연산자	이름	설명	예시 (7, 4)
/	True Division	소수점까지 계산하는 일반 나눗셈	`7 / 4 = 1.75`
//	Floor Division	소수점 아래를 버리고 **몫**만 취함	`7 // 4 = 1`
%	Modulo (Mod)	나눗셈 후 남은 **나머지**를 취함	`7 % 4 = 3`

단위 변환의 마법사: // 와 %

나눗셈의 몫과 나머지를 함께 사용하면 한 단위를 다른 단위로 쉽게 분리할 수 있습니다. 이때 나누는 기준이 되는 수를 **Modulus(법)**라고 합니다.

예시: 센티미터(cm)를 미터(m)로 변환

Modulus = 100 (100cm = 1m)

```
total_cm = 193
meters = total_cm // 100    # 결과: 1 (몫)
rem_cm = total_cm % 100    # 결과: 93 (나머지)
print(meters, "m", rem_cm, "cm")    → 1m 93cm
```

2.6 The Math Module

2.6 내장 함수 vs 모듈 (Built-in vs Module)

파이썬은 효율적인 메모리 관리를 위해 기능을 두 단계로 나누어 제공합니다.

- **내장 함수 (Built-in):** 별도의 준비 없이 즉시 사용 가능 (Ex: `print()`, `input()`, `len()`)
- **모듈 (Module):** 특정 분야의 전문 도구함. 사용 전 반드시 가져오기(`import`)가 필요함

왜 나누어 놓았을까요?

모든 기능을 한꺼번에 불러오면 프로그램이 무거워지기 때문입니다. 필요할 때만 전문 도구함을 책상 위로 가져오는 것과 같습니다.

도구함 가져오기: import 문

전문 도구를 사용하려면 프로그램 최상단에 `import` 명령어를 써주어야 합니다.

사용 법: Dot(.) 연산자

```
import math    # math 도구함을 가져옵니다.
print(math.sqrt(25))    # math '안에 있는' sqrt 기능을 사용!
```

주의: NameError

`import math` 없이 `math.sqrt()`를 쓰면 파이썬은 "math가 뭐야?"라며 에러를 발생시킵니다.

주요 수학 상수와 함수

분류	도구 (명령어)	설명 / 값
상수	`math.pi`	원주율 ($\pi \approx 3.14159$)
상수	`math.e`	자연상수 ($e \approx 2.71828$)
함수	`math.sqrt(x)`	제곱근 ($\sqrt{x}$) $\rightarrow$ **결과는 항상 float**
함수	`math.ceil(x)`	올림 (Ceiling) $\rightarrow$ 예: 7.1 $\rightarrow$ 8
함수	`math.floor(x)`	내림 (Floor) $\rightarrow$ 예: 7.9 $\rightarrow$ 7

실전 예제 1: 두 점 사이의 거리

피타고라스 정리를 이용해 좌표평면 위의 두 점 (x_1, y_1)과 (x_2, y_2) 사이의 거리를 구해봅시다.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Python Code

```
import math
x_dist = (x2 - x1) ** 2
y_dist = (y2 - y1) ** 2
distance = math.sqrt(x_dist + y_dist)
print("The distance is", distance)
```

전문가의 비법: 모르면 검색하라! (Standard Library)

- **Standard Library (표준 라이브러리):** 파이썬 설치 시 기본으로 제공되는 "순정 부품" 세트입니다. `math`, `random`, `datetime` 등이 있습니다.
- **검색의 중요성:** 모든 함수를 외울 필요는 없습니다. 필요한 기능이 있을 때 공식 문서를 찾는 능력이 실력입니다.

개발자의 정석적인 검색 프로세스(보통은 구글에서 그냥 검색)

1. **질문 던지기:** "파이썬에서 올림(Ceiling)은 어떻게 하지?"
2. **검색 (Google/AI):** `python math module documentation` 검색
3. **공식 문서 확인:** docs.python.org에서 정확한 함수명과 사용법 확인
4. **코드 적용:** `import math` 후 `math.ceil()` 사용

3.1 Strings Revisited (문자열 심화)

3.1 문자열 인덱싱 (Indexing)

문자열 내의 각 문자는 고유의 번호인 **인덱스(Index)**를 가집니다. 파이썬은 두 가지 방향의 인덱싱을 지원합니다.

- **Forward Indexing:** 0부터 시작 (왼쪽 → 오른쪽)
- **Backward Indexing:** -1부터 시작 (오른쪽 → 왼쪽)

인덱싱 예시: `s = "Python!"`

`s[0]` → `'P'`

`s[-1]` → `'!'` (마지막 글자를 찾을 때 매우 유용!)

`s[1] + s[-2]` → `'yn'`

Unicode와 ASCII: 문자의 숫자 표현

컴퓨터는 문자를 직접 저장하지 않고, 각 문자에 **할당된 숫자**를 저장합니다.

- **ASCII:** 초창기 표준 (128개 영문 위주)
- **Unicode:** 전 세계 모든 언어를 포함하는 현대 표준 (100만 개 이상)

함수	역할	예시
`ord(char)`	문자를 숫자로 변환 (Order)	`ord("A")` → 65
`chr(int)`	숫자를 문자로 변환 (Character)	`chr(33)` → `"!"`

이스케이프 시퀀스 (Escape Sequences)

백슬래시(\)를 사용하여 화면에 직접 타이핑하기 어려운 특수 문자를 표현합니다.

기호	의미	출력 예시
`\n`	줄바꿈 (Newline)	`"Hello\nWorld"` → 두 줄로 출력
`\t`	탭 (Tab)	문장 사이에 일정한 간격 추가
`\'`	작은따옴표	문자열 내에서 ' 출력
`\\`	백슬래시	\ 기호 자체를 출력

Concept Check: 문자열과 유니코드

Q1. 인덱싱 결합 (Indexing & Concatenation)

`s = "Python!"` 일 때, `s[1] + s[-1]`의 값은 무엇일까요?

a. `"P!"`
b. `"y!"`
c. `"yn"`

Q2. 파이썬의 자료형 (Data Types)

`s = "Python!"` 일 때, `s[0]`의 객체 타입(Type)은 무엇일까요?

a. `character` (문자형)
b. `integer` (정수형)
c. `string` (문자열)

Any Questions?